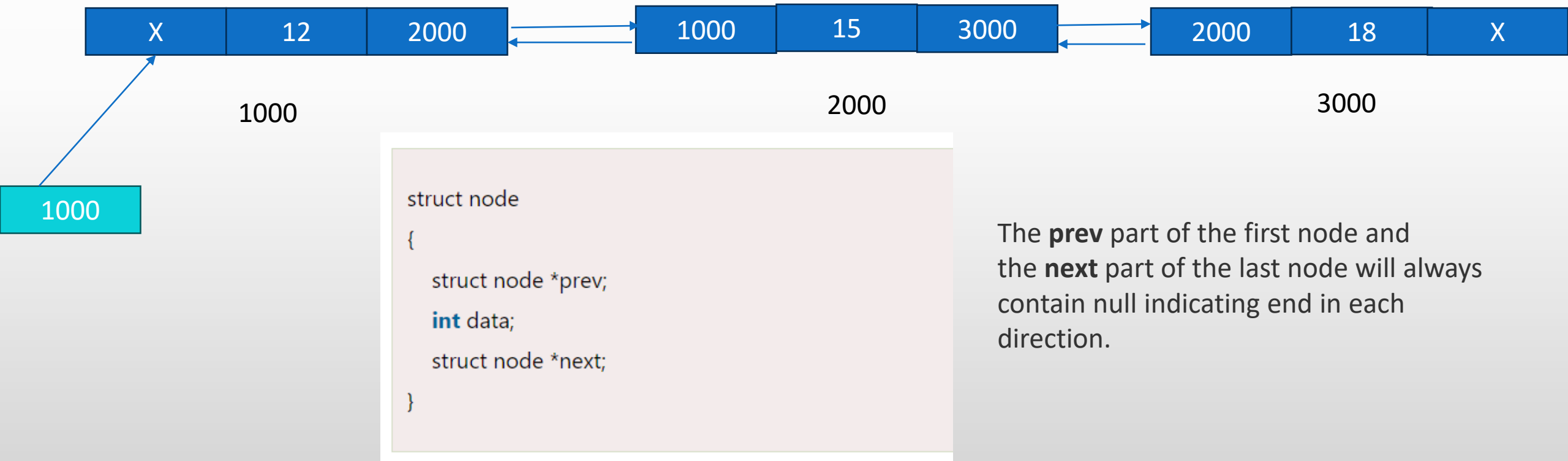


Data Structure & Algorithms

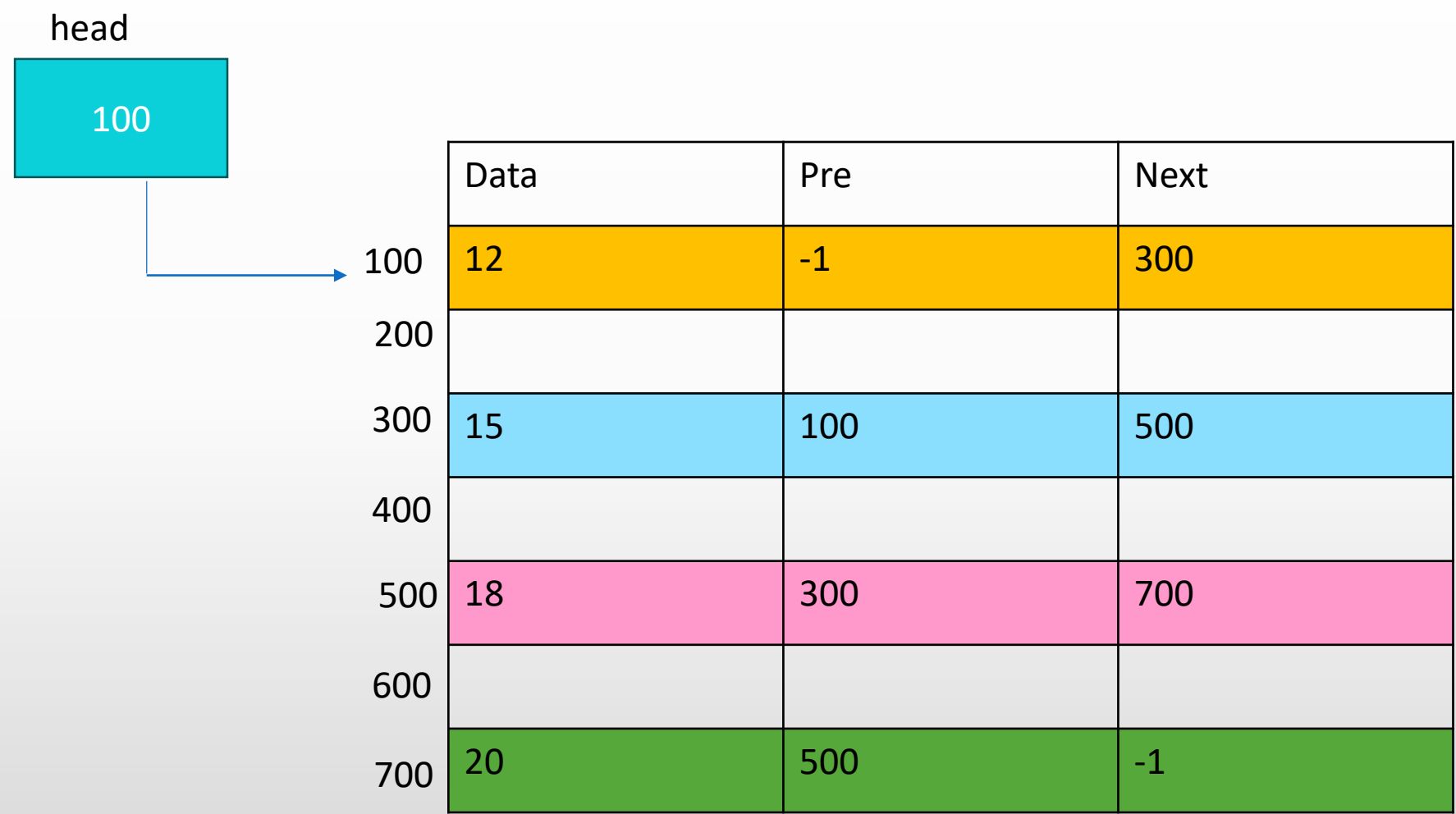
Doubly Linked List

Introduction

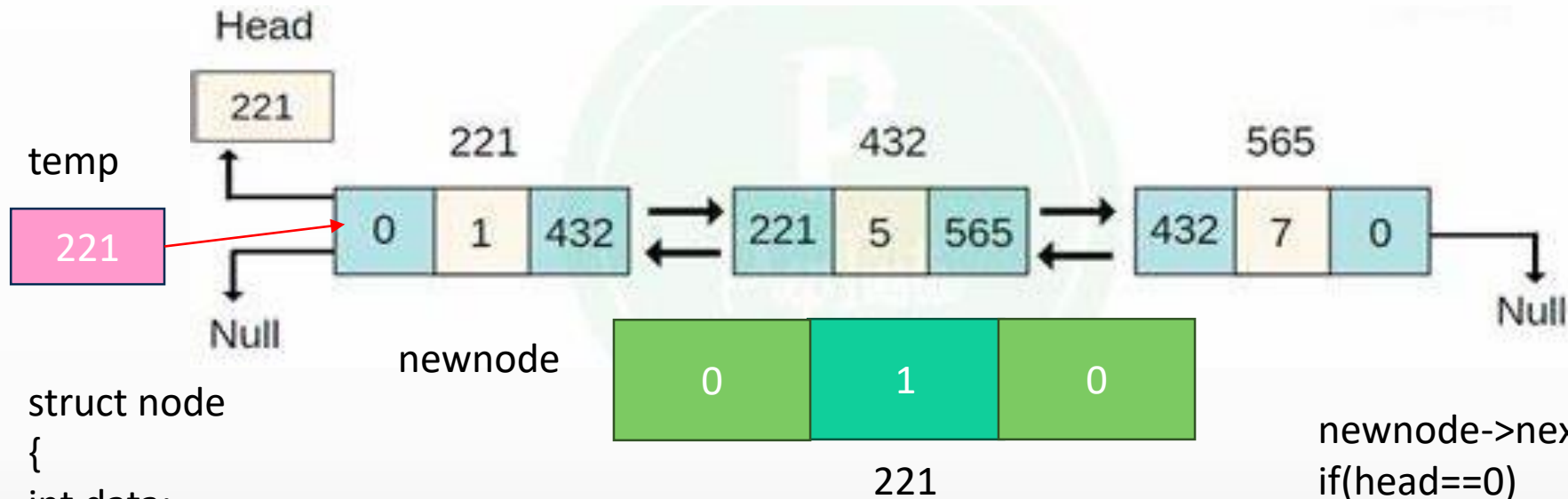
Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer) , pointer to the previous node (previous pointer).



Memory Representation of a doubly linked list



C Implementation



```
struct node
{
    int data;
    struct node *next;
    struct node *pre;
};
struct node *head, *newnode, *temp;
head=0;
newnode =(structnode*)malloc(sizeof(struct node))
printf("enter data");
scanf("%d", &newnode -> data);
newnode ->pre=0;
```

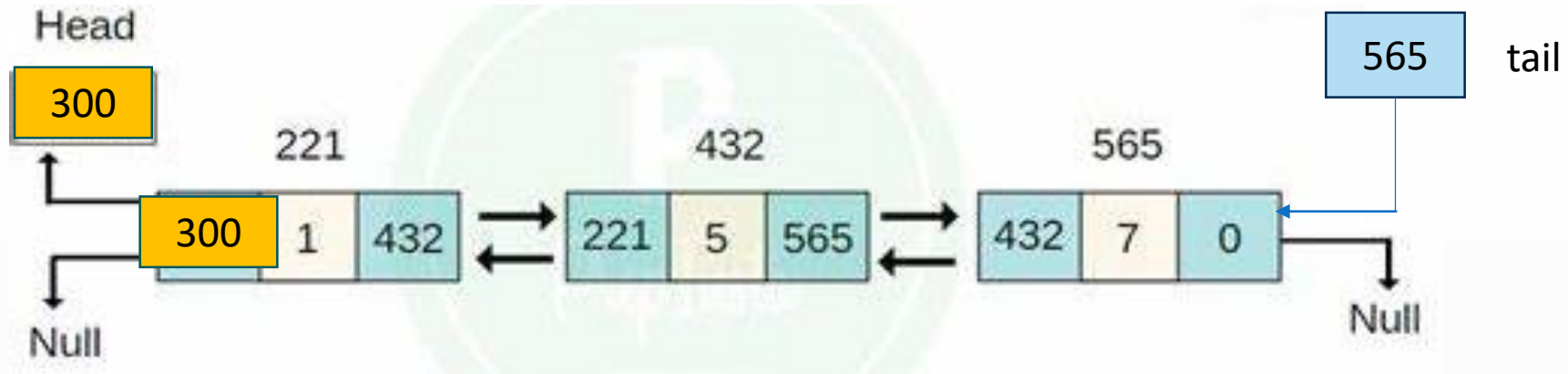
```
newnode->next=0;
if(head==0)
{
    head=temp=newnode;}
else
{
    temp->next=newnode;
    newnode->pre=temp;
    temp=newnode;
}
```

Insertion in DLL

The insertion into a doubly linked list can be performed at different positions. Based on the position of the new node being inserted, the insertion is categorized into the following categories.

1.	Insertion at beginning	It involves inserting any element at the front of the list.
2.	Insertion at end of the list	It involves insertion at the last of the linked list. The new node can be inserted as the only node in the list or it can be inserted as the last one.
3.	Insertion after specified node	It involves insertion after the specified node of the linked list.

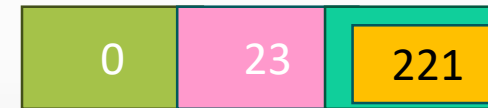
Insertion at the beginning



```
struct node
```

```
{  
    int data;  
    struct node *next;  
    struct node *pre;  
};  
struct node *head, *newnode, *tail;  
head=0;  
    newnode =(structnode*)malloc(sizeof(struct node))  
printf("enter data");  
    scanf("%d", &newnode -> data);  
newnode ->pre=0;  
newnode->next=o;
```

newnode



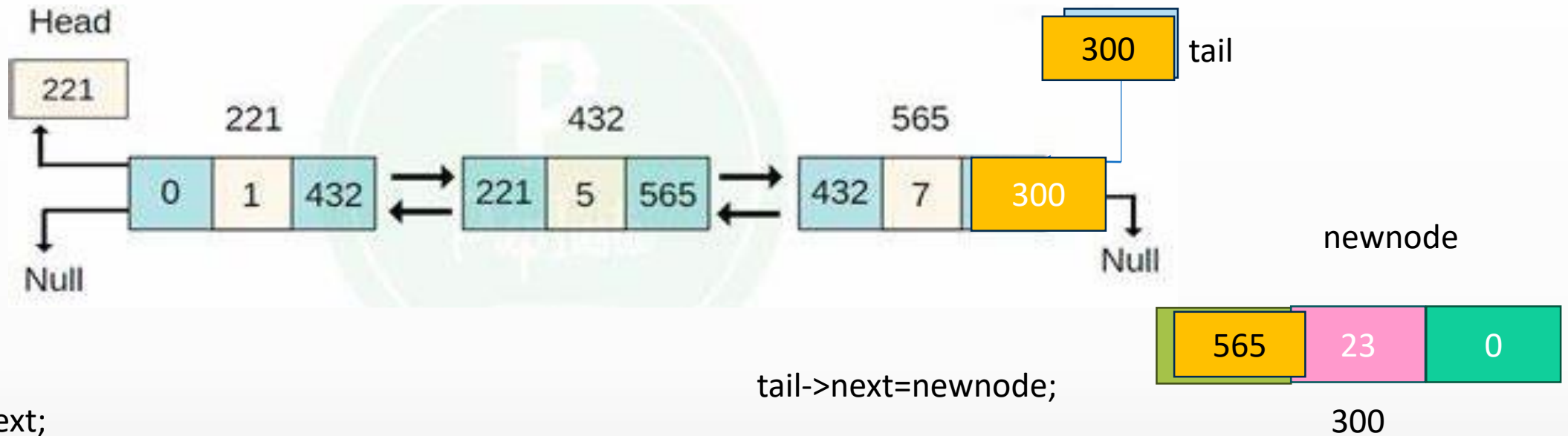
300

```
head->prev=newnode;
```

```
newnode->next=head;
```

```
head=newnode;
```

Insertion at the end

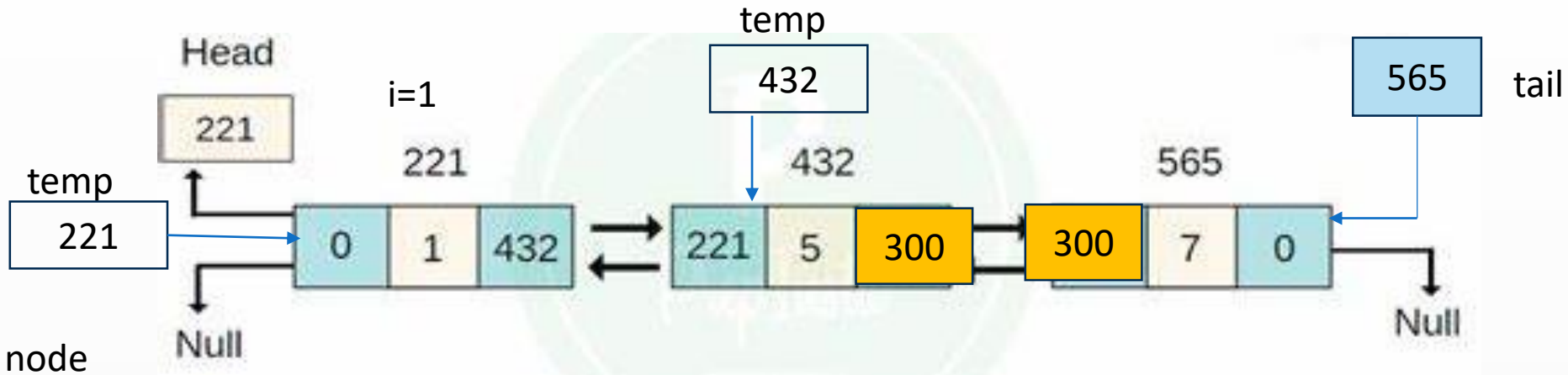


```
struct node
{
    int data;
    struct node *next;
    struct node *pre;
};
struct node *head, *newnode, *tail;
head=0;
    newnode =(structnode*)malloc(sizeof(struct node))
printf("enter data");
    scanf("%d", &newnode -> data);
newnode ->pre=0;
newnode->next=o;
```

```
tail->next=newnode;
```

```
newnode->prev=tail;
tail=newnode;
}
```

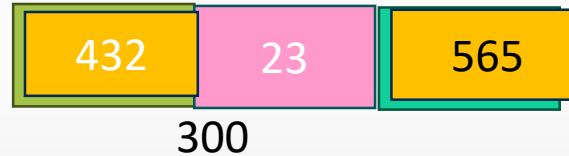
Insertion at specified position



struct node

```
{
int data;
struct node *next;
struct node *pre;
};
void insertatPos()
struct node *head, *newnode,*tail;
int pos,i=1,l;
printf("enter pos");
scanf("%d", &pos);
```

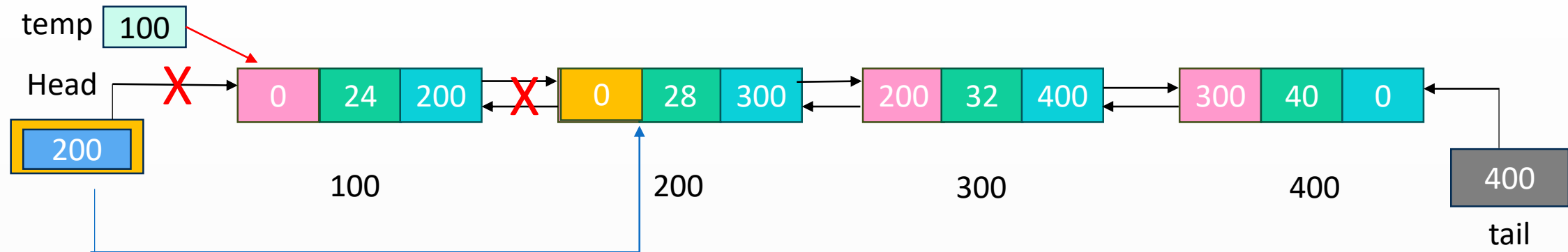
newnode



```
temp=head;
while(i<pos)
{
temp=temp->next;
i++;
}
```

```
newnode->prev=temp;
newnode->next=temp->next;
temp->next->prev=newnode;
temp->next=newnode;
```

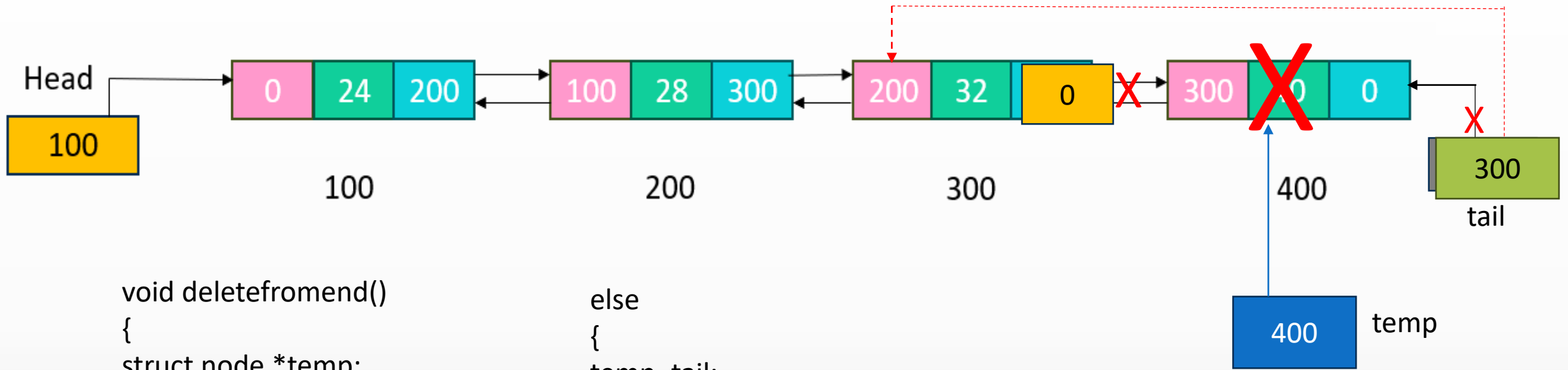

Deletion from Beginning



```
void deletfromBeg()  
{  
    struct node *temp;  
    if(head==0)  
    {  
        printf("list is empty");  
    }  
}
```

```
else  
{  
    temp=head;  
    head=head->next;  
    head->prev=0;  
    free(temp);  
}
```

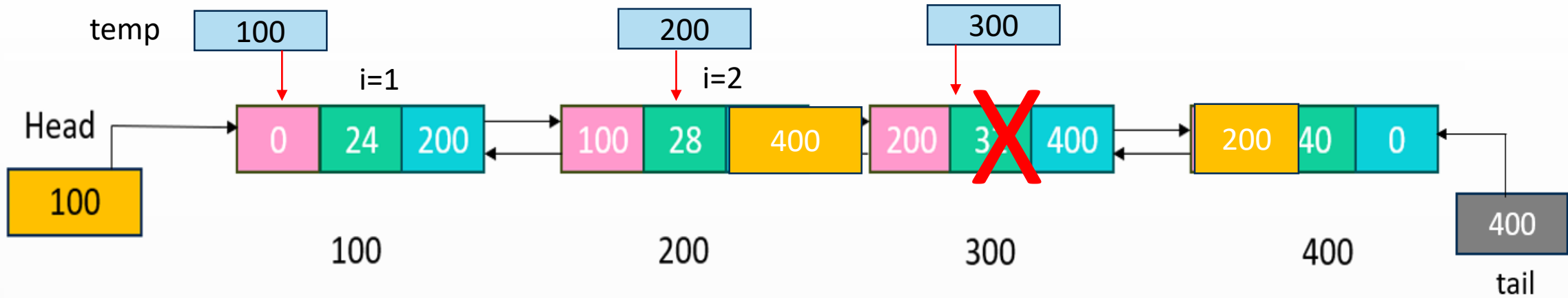
Deletion from end



```
void deletefromend()  
{  
    struct node *temp;  
    if(head==0)  
    {  
        printf("list is empty");  
    }  
}
```

```
else  
{  
    temp=tail;  
    tail->prev->next=0;  
    tail=tail->prev;  
    free(temp);  
}
```

Deletion from specified Position



```
void deletefromspecifiedpos()
{
    int pos,i=1;
    struct node *temp;
    temp=head;
    printf("enter position");    Pos=3
    scanf("%d", &pos);
    while(i<pos)
    {
        temp=temp->next;
        i++;
    }
```

```
{
    temp->prev->next=temp->next;
    temp->next->prev=temp->prev;
    free(temp);
}
```